

FLUJOS DETALLADOS — API ORQUESTADORDB

EPS Familiar de Colombia — Backend FastAPI que orquesta operaciones entre PostgreSQL (ERP Messiah, esquema administrativo) y SQL Server (OrquestacionDB).

Por cada endpoint se documentan: tablas PostgreSQL y SQL Server (lectura/escritura), validaciones de negocio y seguridad, flujo paso a paso. Radicación SIIFA: scripts externos (sin endpoint API).

1. Convenciones transversales

Autenticación JWT en todas las rutas excepto POST /auth/login.

Lista blanca: seg.usuario_endpoints (método + ruta) para usuarios no administradores.

Auditoría API: orq.log_procesos con servicio lógico, usuario, documento, resultado y detalle JSON.

Esquema PostgreSQL principal: administrativo.* (Messiah ERP).

2. Índice de endpoints

Grupo	Método	Ruta	Resumen
Auth	POST	/auth/login	Inicio de sesión y emisión de JWT.
Auth	GET	/auth/me	Perfil del usuario autenticado y permisos seg.*.
Health	GET	/health	Verificación de disponibilidad del servicio.
Procesos Afiliado	POST	/consultas/portabilidad	Historial completo de portabilidad del afiliado (CA1).
Procesos Afiliado	POST	/consultas/pqr/pqr-afiliado	Listado de PQR del afiliado (POST, filtro opcional por radicado).
Procesos Afiliado	GET	/consultas/afiliado/portabilidad	Detalle de portabilidad



			del afiliado (GET).
Procesos Afiliado	GET	/consultas/afiliado/pqr	Consulta PQR del afiliado (GET, consecutivo opcional).
Procesos Afiliado	POST	/consultas/afiliado	Consulta general de datos del afiliado.
Procesos Afiliado	POST	/consultas/certificado-afiliacion	Generación de certificado de afiliación en PDF (Base64).
Procesos Afiliado	POST	/afiliados/actualizacion-datos-micrositio	Registro de solicitud de actualización de datos desde micrositio.
Procesos IPS	POST	/agendamientos	Crear cita médica para afiliado activo.
Procesos IPS	PUT	/agendamientos/{id_agendamento}	Actualizar estado de una cita existente.
Procesos IPS	POST	/historia_clinica	Registrar historia clínica ambulatoria (JSON Version_12.0).
Procesos IPS	POST	/historia_clinica/hl7	Registrar historia clínica desde Bundle FHIR + PDF soporte.
Procesos IPS	POST	/afiliados/autorizacion-orden-medica-ips	Paso 1 — Solicitud y autorización emitida de medicamentos por orden médica IPS.
Procesos IPS	POST	/afiliados/autorizacion-orden-medica-ips/activar	Paso 2 — Activación de

			autorización IPS con PIN.
Procesos Financieros	POST	/execute/facturas_decimales	Ajuste decimal de facturas (proceso financiero batch).
Procesos Financieros	POST	/execute/update_saldo_y_valor_factura	Actualización unificada de saldo y valor por aplicar de facturas.
Ejecución (legacy)	POST	/execute/service1	Alias legacy del ajuste decimal (oculto en OpenAPI).

3. Procesos SIIFA

Sin endpoints API vigentes. Radicación vía:

- export/siifa_radificacion_sin_seguimiento_api.py
- scripts/siifa_radificacion_sync.py

Tablas típicas: dbo.SIIFA_* (SQL Server), administrativo.rips_af / rips_resumen (PostgreSQL).

4. Flujos por endpoint

Auth

POST /auth/login

Inicio de sesión y emisión de JWT.

Grupo OpenAPI

Autenticación

Autorización adicional

Content-Type

Auditoría

Auth

Público (sin Bearer).

Ninguno.

application/json

orq.log_accesos (SUCCESS/ERROR, IP, mensaje).

Tablas PostgreSQL (ERP Messiah)

— (sin acceso directo)

Tablas SQL Server (OrquestacionDB)

Lectura: orq.usuarios, seg.perfiles,

seg.usuario_perfil

Escritura: orq.log_accesos, orq.usuarios (ultimo_login)

Validaciones

- Pydantic: username y password obligatorios.
- Usuario debe existir y estar activo (orq.usuarios.activo = 1).

- Contraseña: hash SHA-256 hex o texto plano transitorio (verify_password_hash).
- IP del cliente autorizada según orq.usuarios.IpPermitida (si aplica).
- Roles desde seg.perfiles; si usuarios.tipo = 1 se agrega rol ADMINISTRADOR.

Entrada

JSON: `{ "username": "api_user", "password": "****" }`

Salida

200 — TokenResponse: access_token (JWT), expires_in (segundos).

Flujo detallado

1. Resolver IP del cliente.
2. Buscar usuario activo en orq.usuarios.
3. Verificar contraseña; si falla → log ERROR → 401.
4. Validar lista blanca IP.
5. Actualizar ultimo_login y registrar log SUCCESS.
6. Emitir JWT con roles y devolver token.

Códigos de respuesta

200 | 401 credenciales inválidas o usuario inactivo | 403 IP no autorizada | 422

GET /auth/me

Perfil del usuario autenticado y permisos seg.*.

Grupo OpenAPI	Auth
Autenticación	Bearer JWT.
Autorización adicional	Lista blanca seg.usuario_endpoints si aplica.
Content-Type	—
Auditoría	No registra orq.log_procesos.

Tablas PostgreSQL (ERP Messiah)

— (sin acceso directo)

Tablas SQL Server (OrquestacionDB)

Lectura: orq.usuarios, seg.modulos, seg.acciones, seg.perfiles, seg.permisos, seg.usuario_perfil, seg.usuario_endpoints

Validaciones

- Bearer JWT válido (iss, aud, exp, sub, jti).
- Usuario activo en orq.usuarios.
- IP del cliente dentro de IpPermitida del usuario (si está configurada).
- Lista blanca seg.usuario_endpoints para usuarios no administradores (si tienen reglas activas).
- Si usuarios.tipo = 1: todos los módulos activos con todas las acciones.
- Si no admin: módulos/acciones desde seg.permisos con permitido = true.
- 503 si no hay permisos SELECT sobre esquema seg.

Entrada

Header Authorization: Bearer <token>.

Salida

200 — MeResponse: username, es_administrador, modulos[].

Flujo detallado

1. Validar JWT y usuario activo.
2. Consultar get_me en SqlServerRepository.
3. Armar listado de módulos y acciones.
4. Devolver MeResponse.

Códigos de respuesta

200 | 401 | 503 error leyendo seg.*

Health

GET /health

Verificación de disponibilidad del servicio.

Grupo OpenAPI

Autenticación

Autorización adicional

Content-Type

Auditoría

Health

Bearer JWT.

Lista blanca seg.usuario_endpoints si aplica.

—

Sin registro de consumo.

Tablas PostgreSQL (ERP Messiah)

— (sin acceso directo)

Tablas SQL Server (OrquestacionDB)

Lectura: orq.usuarios, seg.usuario_endpoints

Validaciones

- Bearer JWT válido (iss, aud, exp, sub, jti).
- Usuario activo en orq.usuarios.
- IP del cliente dentro de IpPermitida del usuario (si está configurada).
- Lista blanca seg.usuario_endpoints para usuarios no administradores (si tienen reglas activas).
- No ejecuta consulta de salud a bases de datos de negocio.

Entrada

Sin cuerpo.

Salida

200 — {"status": "ok"}.

Flujo detallado

1. Validar JWT.
2. Responder status ok.

Códigos de respuesta
200 | 401

Procesos Afiliado

POST /consultas/portabilidad

Historial completo de portabilidad del afiliado (CA1).

Grupo OpenAPI

Autenticación

Autorización adicional

Content-Type

Auditoría

Procesos Afiliado

Bearer JWT.

Lista blanca seg.usuario_endpoints si aplica.

application/json

orq.log_procesos (consumo API, servicio lógico prefijado api:).
servicio=consulta_portabilidad.

Tablas PostgreSQL (ERP Messiah)

Lectura: administrativo.af_afiliado, administrativo.af_afiliado_movilidad,
administrativo.tb_municipio

Tablas SQL Server (OrquestacionDB)

Escritura: orq.log_procesos (consumo API, servicio lógico prefijado api:).

Validaciones

- Bearer JWT válido (iss, aud, exp, sub, jti).
- Usuario activo en orq.usuarios.
- IP del cliente dentro de IpPermitida del usuario (si está configurada).
- Lista blanca seg.usuario_endpoints para usuarios no administradores (si tienen reglas activas).
- tipo_identificacion normalizado vía catálogo (CC, TI, RC, etc.).
- Afiliado debe existir (404).
- CA1: estado_afiliado debe ser Activo (1) o Activo Carnetizado (5); si no → 400.

Entrada

JSON: tipo_identificacion, numero_identificacion.

Salida

200 — PortabilidadConsultaResponse con historial de movimientos.

Flujo detallado

1. Resolver documento.
2. Consultar resumen afiliado en PostgreSQL.
3. Validar estado activo para portabilidad.
4. Obtener movimientos (fetch_todas_portabilidades_por_afiliado).
5. Construir respuesta y registrar consumo.

Códigos de respuesta

200 | 400 afiliado no activo | 404 | 401 | 422 | 500



POST /consultas/pqr/por-afiliado

Listado de PQR del afiliado (POST, filtro opcional por radicado).

Grupo OpenAPI

Autenticación

Autorización adicional

Content-Type

Auditoría

Procesos Afiliado

Bearer JWT.

Lista blanca seg.usuario_endpoints si aplica.

application/json

orq.log_procesos (consumo API, servicio lógico prefijado api:).

servicio=consulta_pqr_por_afiliado.

Tablas PostgreSQL (ERP Messiah)

Lectura: administrativo.af_afiliado, administrativo.pqr_peticion_encabezado, administrativo.pqr_tipo_solicitud, administrativo.prb_dependencia

Tablas SQL Server (OrquestacionDB)

Escritura: orq.log_procesos (consumo API, servicio lógico prefijado api:).

Validaciones

- Bearer JWT válido (iss, aud, exp, sub, jti).
- Usuario activo en orq.usuarios.
- IP del cliente dentro de IpPermitida del usuario (si está configurada).
- Lista blanca seg.usuario_endpoints para usuarios no administradores (si tienen reglas activas).
- Afiliado debe existir (404).
- consecutivo_peticion opcional: si se envía, filtra una PQR; si no existe devuelve lista vacía (200).

Entrada

JSON: tipo_identificacion, numero_identificacion, consecutivo_peticion (opcional).

Salida

200 — PqrAfiliadoListaResponse.

Flujo detallado

1. Verificar afiliado.
2. Listar PQR o buscar por consecutivo.
3. Mapear a resumen y registrar consumo.

Códigos de respuesta

200 | 404 | 401 | 422 | 500

GET /consultas/afiliado/portabilidad

Detalle de portabilidad del afiliado (GET).

Grupo OpenAPI

Autenticación

Procesos Afiliado

Bearer JWT.

Autorización adicional
Content-Type
Auditoría

Lista blanca seg.usuario_endpoints si aplica.

—
orq.log_procesos (consumo API, servicio lógico
prefijado api:).
servicio=consulta_portabilidad_resumen_afiliado.

Tablas PostgreSQL (ERP Messiah)

Lectura: administrativo.af_afiliado, administrativo.af_afiliado_movilidad,
administrativo.ct_ips, administrativo.tb_zonificacion_ips_encabezado,
administrativo.tb_municipio

Tablas SQL Server (OrquestacionDB)

Escritura: orq.log_procesos (consumo API, servicio lógico prefijado api:).

Validaciones

- Bearer JWT válido (iss, aud, exp, sub, jti).
- Usuario activo en orq.usuarios.
- IP del cliente dentro de IpPermitida del usuario (si está configurada).
- Lista blanca seg.usuario_endpoints para usuarios no administradores (si tienen reglas activas).
- Query: tipo_identificacion, numero_identificacion obligatorios.
- Afiliado debe existir (404).
- No aplica filtro de estado activo (a diferencia de POST /consultas/portabilidad).

Entrada

Query: tipo_identificacion, numero_identificacion.

Salida

200 — PortabilidadAfiliadoResponse.

Flujo detallado

1. Verificar afiliado.
2. Consultar detalle portabilidad.
3. Devolver ítems y registrar consumo.

Códigos de respuesta

200 | 404 | 401 | 500

GET /consultas/afiliado/pqr

Consulta PQR del afiliado (GET, consecutivo opcional).

Grupo OpenAPI

Autenticación

Autorización adicional

Content-Type

Auditoría

Procesos Afiliado

Bearer JWT.

Lista blanca seg.usuario_endpoints si aplica.

—
orq.log_procesos (consumo API, servicio lógico
prefijado api:).
servicio=consulta_pqr_afiliado_opcional_consecutivo.

Tablas PostgreSQL (ERP Messiah)

Lectura: administrativo.af_afiliado, administrativo.pqr_peticion_encabezado,
administrativo.prb_dependencia

Tablas SQL Server (OrquestacionDB)

Escritura: orq.log_procesos (consumo API, servicio lógico prefijado api:).

Validaciones

- Bearer JWT válido (iss, aud, exp, sub, jti).
- Usuario activo en orq.usuarios.
- IP del cliente dentro de IpPermitida del usuario (si está configurada).
- Lista blanca seg.usuario_endpoints para usuarios no administradores (si tienen reglas activas).
- Afiliado debe existir (404).
- consecutivo_pqr opcional en query.

Entrada

Query: tipo_identificacion, numero_identificacion; consecutivo_pqr (opcional).

Salida

200 — PqrAfiliadoConsultaResponse.

Flujo detallado

1. Verificar afiliado.
2. Consultar PQR con filtro opcional.
3. Registrar consumo.

Códigos de respuesta

200 | 404 | 401 | 500

POST /consultas/afiliado

Consulta general de datos del afiliado.

Grupo OpenAPI

Autenticación

Autorización adicional

Content-Type

Auditoría

Procesos Afiliado

Bearer JWT.

Lista blanca seg.usuario_endpoints si aplica.

application/json

orq.log_procesos (consumo API, servicio lógico prefijado api:).
servicio=consulta_afiliado.

Tablas PostgreSQL (ERP Messiah)

Lectura: administrativo.af_afiliado, administrativo.af_afiliado_complemento,
administrativo.tb_municipio, administrativo.ct_ips,
administrativo.tb_zonificacion_ips_encabezado, administrativo.af_afiliado_movilidad,
administrativo.ct_ips_contrato

Tablas SQL Server (OrquestacionDB)

Escritura: orq.log_procesos (consumo API, servicio lógico prefijado api:).

Validaciones

- Bearer JWT válido (iss, aud, exp, sub, jti).
- Usuario activo en orq.usuarios.
- IP del cliente dentro de IpPermitida del usuario (si está configurada).
- Lista blanca seg.usuario_endpoints para usuarios no administradores (si tienen reglas activas).
- Afiliado debe existir (404).
- Si nit_ips se envía: debe existir en administrativo.ct_ips (404).
- Si nit_ips: afiliado debe tener departamento_codigo (400).
- Si nit_ips: debe existir contrato IPS para el departamento del afiliado en ct_ips_contrato (404).
- Enriquece respuesta con numero_contrato y estado_contrato.

Entrada

JSON: tipo_identificacion, numero_identificacion, nit_ips (opcional).

Salida

200 — ConsultaAfiliadoResponse.

Flujo detallado

1. Consultar afiliado.
2. Si nit_ips: validar IPS y contrato por departamento.
3. Armar respuesta y registrar consumo.

Códigos de respuesta

200 | 400 | 404 | 401 | 422 | 500

POST /consultas/certificado-afiliacion

Generación de certificado de afiliación en PDF (Base64).

Grupo OpenAPI

Procesos Afiliado

Autenticación

Bearer JWT.

Autorización adicional

Lista blanca seg.usuario_endpoints si aplica.

Content-Type

application/json

Auditoría

orq.log_procesos (consumo API, servicio lógico prefijado api:).
servicio=generacion_certificado_afiliacion.

Tablas PostgreSQL (ERP Messiah)

Lectura: administrativo.af_afiliado, administrativo.af_afiliado_complemento,
administrativo.ct_ips, administrativo.tb_zonificacion_ips_encabezado,
administrativo.af_afiliado_ingreso_contributivo, administrativo.af_aportante

Tablas SQL Server (OrquestacionDB)

Escritura: orq.log_procesos (consumo API, servicio lógico prefijado api:).

Validaciones

- Bearer JWT válido (iss, aud, exp, sub, jti).
- Usuario activo en orq.usuarios.
- IP del cliente dentro de IpPermitida del usuario (si está configurada).
- Lista blanca seg.usuario_endpoints para usuarios no administradores (si tienen reglas activas).
- Afiliado debe existir (404).
- No valida estado activo en código (genera PDF con datos del registro).
- Régimen contributivo: incluye relaciones laborales (af_afiliado_ingreso_contributivo, af_aportante).
- Plantilla DOCX según régimen; conversión PDF vía LibreOffice (503 si no disponible).

Entrada

JSON: tipo_identificacion, numero_identificacion.

Salida

200 — CertificadoAfiliacionResponse con archivo_pdf_base64.

Flujo detallado

1. Obtener datos afiliado para certificado.
2. Cargar relaciones laborales si contributivo.
3. Rellenar plantilla DOCX y convertir a PDF.
4. Codificar Base64 y registrar consumo.

Códigos de respuesta

200 | 404 | 503 LibreOffice/plantilla | 500 | 401 | 422

POST /afiliados/actualizacion-datos-micrositio

Registro de solicitud de actualización de datos desde micrositio.

Grupo OpenAPI

Autenticación

Autorización adicional

Content-Type

Auditoría

Procesos Afiliado

Bearer JWT.

Lista blanca seg.usuario_endpoints si aplica.

multipart/form-data

orq.log_procesos (consumo API, servicio lógico prefijado api:).
servicio=actualizacion_datos_micrositio.

Tablas PostgreSQL (ERP Messiah)

Lectura: administrativo.af_afiliado,

administrativo.tb_tipo_soporte

Escritura:

administrativo.af_ticket_administrativo_aseguramiento,



administrativo.af_ticket_administrativo_aseguramiento_detalle,
administrativo.af_ticket_administrativo_aseguramiento_soporte

Tablas SQL Server (OrquestacionDB)

Escritura: orq.log_procesos (consumo API, servicio lógico prefijado api:).

Validaciones

- Bearer JWT válido (iss, aud, exp, sub, jti).
- Usuario activo en orq.usuarios.
- IP del cliente dentro de IpPermitida del usuario (si está configurada).
- Lista blanca seg.usuario_endpoints para usuarios no administradores (si tienen reglas activas).
- barrio, direccion, celular, correo_electronico, observacion obligatorios (422).
- Afiliado debe existir (404) y tener edad ≥ 18 años (400).
- telefono/celular: 10 dígitos; correo con formato válido (validar_dato_contacto).
- Soporte opcional: tipo en {CN, RC, TI, CC, CE, PA, CD, SC, PE, MS, CERTIFICADO_REGISTRADURIA}.
- Soporte: extensión .pdf, .png, .jpg, .jpeg; tamaño máx. TICKET_SUPPORT_MAX_MB; no vacío.
- tipo_soporte_documento sin archivo $\rightarrow$ 400.
- Ticket: tipo_proceso=1, origen_solicitud=2 (micrositio).
- Archivo soporte guardado en disco (ticket_supports_dir), no en blob de BD.

Entrada

Form: tipo_identificacion, numero_identificacion, barrio, direccion, celular, correo_electronico, observacion; telefono (opc.); soporte_documento (opc.).

Salida

200 — ActualizacionDatosMicrositioResponse con consecutivo_ticket.

Flujo detallado

1. Validar campos y soporte opcional.
2. Verificar afiliado mayor de edad.
3. Validar formatos de contacto.
4. Crear ticket en PostgreSQL.
5. Registrar consumo.

Códigos de respuesta

200 | 400 | 404 | 422 | 401 | 500

Procesos IPS

POST /agendamientos

Crear cita médica para afiliado activo.

Grupo OpenAPI

Autenticación

Procesos IPS

Bearer JWT.

Autorización adicional

Content-Type

Auditoría

Lista blanca seg.usuario_endpoints si aplica.

application/json

orq.log_procesos (consumo API, servicio lógico prefijado api:).
servicio=crear_agendamiento.

Tablas PostgreSQL (ERP Messiah)

Lectura: administrativo.af_afiliado

Tablas SQL Server (OrquestacionDB)

Lectura:

orq.agendamiento

Escritura: orq.agendamiento, orq.log_procesos (consumo API, servicio lógico prefijado api:).

Validaciones

- Bearer JWT válido (iss, aud, exp, sub, jti).
- Usuario activo en orq.usuarios.
- IP del cliente dentro de IpPermitida del usuario (si está configurada).
- Lista blanca seg.usuario_endpoints para usuarios no administradores (si tienen reglas activas).
- AgendamientoRequest: longitudes de campos y estado 0-5.
- hora_cita formato HH:MM[:SS]; fecha_cita + hora_cita >= ahora (400).
- Afiliado estado_afiliado IN (1, 5) — activo (400).
- Sin cita duplicada: misma sede, profesional, fecha, hora, especialidad, estado (409).

Entrada

JSON: sede, tipoDoc, numDoc, tipoDoc_Prof, numDoc_Prof, fecha_cita, hora_cita, especialidad, estado, usuario_asignacion (opc.), programa (opc.).

Salida

201 — AgendamientoResponse con id_agendamiento.

Flujo detallado

1. Validar fecha/hora.
2. Verificar afiliado activo en PostgreSQL.
3. Comprobar conflicto en orq.agendamiento.
4. Insertar cita y registrar consumo.

Códigos de respuesta

201 | 400 | 409 | 401 | 500

PUT /agendamientos/{id_agendamiento}

Actualizar estado de una cita existente.

Grupo OpenAPI

Autenticación

Procesos IPS

Bearer JWT.

Autorización adicional

Content-Type

Auditoría

Lista blanca seg.usuario_endpoints si aplica.

application/json

orq.log_procesos (consumo API, servicio lógico prefijado api:).
servicio=editar_agendamiento.

Tablas PostgreSQL (ERP Messiah)

— (sin acceso directo)

Tablas SQL Server (OrquestacionDB)

Lectura:

orq.agendamiento

Escritura: orq.agendamiento, orq.log_procesos (consumo API, servicio lógico prefijado api:).

Validaciones

- Bearer JWT válido (iss, aud, exp, sub, jti).
- Usuario activo en orq.usuarios.
- IP del cliente dentro de IpPermitida del usuario (si está configurada).
- Lista blanca seg.usuario_endpoints para usuarios no administradores (si tienen reglas activas).
- estado debe ser 0-5 (pendiente, confirmada, cancelada, atendida, no asistió, reprogramada).
- id_agendamiento debe existir (404).

Entrada

Path: id_agendamiento. JSON: estado.

Salida

200 — AgendamientoResponse.

Flujo detallado

1. Actualizar estado en orq.agendamiento.
2. Si no afecta filas → 404.
3. Registrar consumo.

Códigos de respuesta

200 | 400 | 404 | 401 | 500

POST /historia_clinica

Registrar historia clínica ambulatoria (JSON Version_12.0).

Grupo OpenAPI

Procesos IPS

Autenticación

Bearer JWT.

Autorización adicional

Lista blanca seg.usuario_endpoints si aplica.

Content-Type

application/json

Auditoría

orq.log_procesos (consumo API, servicio lógico prefijado api:).
servicio=registrar_historia_clinica.

Tablas PostgreSQL (ERP Messiah)

Lectura: administrativo.af_afiliado, administrativo.ct_ips, administrativo.tb_cie10, administrativo.tb_cup

Tablas SQL Server (OrquestacionDB)

Escritura: orq.historia_clinica, orq.historia_clinica_actividad, orq.log_procesos (consumo API, servicio lógico prefijado api:).

Validaciones

- Bearer JWT válido (iss, aud, exp, sub, jti).
- Usuario activo en orq.usuarios.
- IP del cliente dentro de IpPermitida del usuario (si está configurada).
- Lista blanca seg.usuario_endpoints para usuarios no administradores (si tienen reglas activas).
- Afiliado activo (estado 1 o 5) (404).
- NIT prestador debe existir en administrativo.ct_ips (404).
- EntidadResponsable.Codigo debe ser CCF033 o CCFC33 (400).
- CodigoDiagnosticoPrincipal en tb_cie10 (404).
- CodigoCups en tb_cup (404).
- Estructura Pydantic HistoriaClinicaRequest (Usuario, Prestador, Cita, Mediciones, Actividades).

Entrada

JSON HistoriaClinicaRequest (Version_12.0).

Salida

201 — HistoriaClinicaResponse con nueva_historia_id.

Flujo detallado

1. Validar afiliado, IPS, CIE-10 y CUPS.
2. Insertar orq.historia_clinica y actividades.
3. Registrar consumo.

Códigos de respuesta

201 | 400 | 404 | 401 | 500

POST /historia_clinica/hl7

Registrar historia clínica desde Bundle FHIR + PDF soporte.

Grupo OpenAPI

Autenticación

Autorización adicional

Content-Type

Procesos IPS

Bearer JWT.

Lista blanca seg.usuario_endpoints si aplica.

application/json



Auditoría

orq.log_procesos (consumo API, servicio lógico prefijado api:).
servicio=registrar_historia_clinica_hl7_fhir.

Tablas PostgreSQL (ERP Messiah)

Lectura: administrativo.af_afiliado, administrativo.ct_ips, administrativo.tb_cie10, administrativo.tb_cup

Tablas SQL Server (OrquestacionDB)

Escritura: orq.historia_clinica, orq.historia_clinica_actividad, orq.log_procesos (consumo API, servicio lógico prefijado api:).

Validaciones

- Bearer JWT válido (iss, aud, exp, sub, jti).
- Usuario activo en orq.usuarios.
- IP del cliente dentro de IpPermitida del usuario (si está configurada).
- Lista blanca seg.usuario_endpoints para usuarios no administradores (si tienen reglas activas).
- payload.bundle debe ser objeto Bundle FHIR (400).
- historia_clinica_pdf_base64 obligatorio; Base64 válido; contenido debe iniciar con %PDF- (400).
- Errores de mapeo FHIR → HistoriaClinicaRequest (400).
- Luego aplica todas las validaciones de POST /historia_clinica.
- PDF no se persiste en BD; solo se registra tamaño en detalle de consumo.

Entrada

JSON: bundle (FHIR), historia_clinica_pdf_base64.

Salida

201 — HistoriaClinicaResponse (igual que /historia_clinica).

Flujo detallado

1. Validar bundle y PDF Base64.
2. Mapear FHIR a HistoriaClinicaRequest.
3. Delegar en registrar_historia_clinica.
4. Registrar consumo FHIR.

Códigos de respuesta

201 | 400 | 404 | 401 | 500

POST /afiliados/autorizacion-orden-medica-ips

Paso 1 — Solicitud y autorización emitida de medicamentos por orden médica IPS.

Grupo OpenAPI

Procesos IPS

Autenticación

Bearer JWT.

Autorización adicional

orq.usuarios.autoriza_med = 1
(require_autoriza_med).

Content-Type

multipart/form-data

Auditoría

orq.log_procesos (consumo API, servicio lógico prefijado api:).
servicio=autorizacion_orden_medica_ips.

Tablas PostgreSQL (ERP Messiah)

Lectura: administrativo.af_afiliado, administrativo.af_afiliado_complemento, administrativo.tb_municipio, administrativo.ct_ips, administrativo.ct_ips_sede, administrativo.tb_medicamento, administrativo.tb_unidad_medida, administrativo.ct_ips_contrato, administrativo.ct_ips_contrato_cobertura, administrativo.tb_tarifario_propio_detalle, administrativo.tb_direccionamiento_autorizacion, administrativo.tb_direccionamiento_autorizacion_medicamento, administrativo.tb_medico_solicitante, administrativo.tb_especialidad, administrativo.tb_cie10, administrativo.tb_modalidad_servicio_salud, administrativo.ss_autorizacion, administrativo.ss_autorizacion_medicamento, administrativo.tb_tipo_soporte, administrativo.ss_solicitud
Escritura: administrativo.ct_ips_ss_solicitud, administrativo.tb_parametro, administrativo.ss_solicitud, administrativo.ss_solicitud_atencion, administrativo.ss_solicitud_medicamento, administrativo.ss_autorizacion, administrativo.ss_autorizacion_medicamento, administrativo.ct_ips_ss_solicitud_autorizacion, administrativo.ss_solicitud_soporte

Tablas SQL Server (OrquestacionDB)

Lectura: orq.usuarios (autoriza_med)
Escritura: orq.log_procesos (consumo API, servicio lógico prefijado api:).

Validaciones

- Bearer JWT válido (iss, aud, exp, sub, jti).
- Usuario activo en orq.usuarios.
- IP del cliente dentro de IpPermitida del usuario (si está configurada).
- Lista blanca seg.usuario_endpoints para usuarios no administradores (si tienen reglas activas).
- orq.usuarios.autoriza_med = true (403).
- medicamentos_json: array JSON válido de objetos con CUM, cantidad, días (400).
- Fechas solicitud no futuras (fecha_solicitud_proceso, fecha_solicitud_medico).
- observacion obligatoria; diagnóstico principal CIE-10; registro_profesional en tb_medico_solicitante.
- Sin CUMs duplicados en el listado.
- Afiliado estado_afiliado = 1 (Activo) (400).
- NIT IPS solicitante y direccionamiento deben resolverse en ct_ips (404).
- email obligatorio (formulario o sede/afiliado/IPS); telefono y celular 10 dígitos.
- soporte_orden_medica PDF obligatorio; validación %PDF- y tamaño máximo.
- Origen: Orden médica; atención Enfermedad General; modalidad Ambulatorios.

- Por medicamento: sw_activo, tarifario municipio, contrato vigente, direccionamiento IPS.
- Autorización estado EMITIDA con PIN; autorizacion_activa=false hasta activación.

Entrada

multipart/form-data: documento afiliado, diagnósticos, NIT IPS, fechas, medicamentos_json, soporte_orden_medica PDF, contacto.

Salida

200 — AutorizacionOrdenMedicalpsResponse con PIN y listados de medicamentos.

Flujo detallado

1. Validar autoriza_med y parsear formulario.
2. Validar afiliado, médico, IPS y medicamentos.
3. Crear ss_solicitud y evaluar reglas por CUM.
4. Emitir ss_autorizacion con PIN si aplica.
5. Opcional: PDF JasperStarter.
6. Registrar traza en orq.log_procesos.

Códigos de respuesta

200 | 400 | 403 | 404 | 401 | 422 | 500

POST /afiliados/autorizacion-orden-medica-ips/activar

Paso 2 — Activación de autorización IPS con PIN.

Grupo OpenAPI	Procesos IPS
Autenticación	Bearer JWT.
Autorización adicional	orq.usuarios.autoriza_med = 1.
Content-Type	multipart/form-data
Auditoría	orq.log_procesos (consumo API, servicio lógico prefijado api:).
	servicio=activacion_autorizacion_orden_medica_ips.

Tablas PostgreSQL (ERP Messiah)

Lectura: administrativo.ct_ips, administrativo.tb_municipio, administrativo.ss_autorizacion, administrativo.ss_solicitud, administrativo.af_afiliado
Escritura: administrativo.ss_autorizacion (fecha_activacion, estado_trazabilidad, sw_activo)

Tablas SQL Server (OrquestacionDB)

Lectura: orq.usuarios (autoriza_med)
Escritura: orq.log_procesos (consumo API, servicio lógico prefijado api:).

Validaciones

- Bearer JWT válido (iss, aud, exp, sub, jti).
- Usuario activo en orq.usuarios.
- IP del cliente dentro de IpPermitida del usuario (si está configurada).

- Lista blanca seg.usuario_endpoints para usuarios no administradores (si tienen reglas activas).
- orq.usuarios.autoriza_med = true (403).
- pin_activacion obligatorio (comparación sin distinguir mayúsculas).
- NIT IPS direccionamiento debe resolverse (404).
- Autorización pendiente: documento + PIN + IPS + fecha_activacion IS NULL (404).
- Si ya activada: respuesta 200 idempotente.
- UPDATE solo cuando fecha_activacion IS NULL.

Entrada

Form: tipo_identificacion, numero_identificacion, pin_activacion,
nit_ips_direccionamiento.

Salida

200 — ActivacionOrdenMedicalIpsResponse.

Flujo detallado

1. Validar autoriza_med.
2. Buscar autorización pendiente por documento, PIN e IPS.
3. Activar en ss_autorizacion (estado trazabilidad ACTIVADA).
4. Opcional: PDF autorización.
5. Registrar consumo.

Códigos de respuesta

200 | 403 | 404 | 409 | 401 | 500

Procesos Financieros

POST /execute/facturas_decimales

Ajuste decimal de facturas (proceso financiero batch).

Grupo OpenAPI

Autenticación

Autorización adicional

Content-Type

Auditoría

Procesos Financieros

Bearer JWT.

Módulo «Facturas decimales», acción
EJECUTAR (seg.permisos) o admin.

— (sin cuerpo)

orq.log_procesos

servicio=facturas_decimales.

Tablas PostgreSQL (ERP Messiah)

Lectura: administrativo.sc_saldo_encabezado, administrativo.sc_saldo_detalle,
administrativo.sc_cuenta, administrativo.sc_tercero,
administrativo.sc_factura_encabezado, administrativo.ct_ips

Tablas SQL Server (OrquestacionDB)

Lectura: orq.usuarios, seg.permisos, seg.perfiles, seg.usuario_perfil, seg.modulos,
seg.acciones, orq.resultados_procesos

Escritura: orq.resultados_procesos, orq.log_procesos

Validaciones

- Bearer JWT válido (iss, aud, exp, sub, jti).
- Usuario activo en orq.usuarios.
- IP del cliente dentro de IpPermitida del usuario (si está configurada).
- Lista blanca seg.usuario_endpoints para usuarios no administradores (si tienen reglas activas).
- Permiso módulo «Facturas decimales» + acción «EJECUTAR» (403); admin tipo=1 omite.
- Consulta candidatos: delta saldo $\neq$ 0, saldo_factura > 0, cuenta clase_b=1.
- Tercero NIT coincide con ct_ips.nit; fechas factura 2022-2030.
- Omite referencias ya registradas en orq.resultados_procesos (tipo_proceso=facturas_decimales).

Entrada

Sin cuerpo. Solo Authorization Bearer.

Salida

200 — ApiExecutionResponse: total, success, errors, detail.

Flujo detallado

1. Validar permiso Facturas decimales.
2. Consultar candidatos en PostgreSQL.
3. Persistir en orq.resultados_procesos evitando duplicados.
4. Registrar log y devolver estadísticas.

Códigos de respuesta

200 | 403 | 401 | 500 | 503

POST /execute/update_saldo_y_valor_factura

Actualización unificada de saldo y valor por aplicar de facturas.

Grupo OpenAPI

Autenticación

Autorización adicional

Content-Type

Auditoría

Procesos Financieros

Bearer JWT.

Módulo «Actualizacion saldo y valor»,
acción EJECUTAR o admin.

— (sin cuerpo)

orq.log_procesos

servicio=update_saldo_y_valor_factura.

Tablas PostgreSQL (ERP Messiah)

Lectura: administrativo.sc_saldo_encabezado, administrativo.sc_saldo_detalle,
administrativo.sc_cuenta, administrativo.sc_tercero,
administrativo.sc_factura_encabezado, administrativo.ct_ips
Escritura: administrativo.sc_factura_encabezado (saldo_factura),
administrativo.sc_factura_detalle_valor (valor_por_aplicar)

Tablas SQL Server (OrquestacionDB)

Lectura: orq.usuarios, seg.permisos, seg.perfiles, seg.usuario_perfil, seg.modulos, seg.acciones

Escritura: orq.log_procesos

Validaciones

- Bearer JWT válido (iss, aud, exp, sub, jti).
- Usuario activo en orq.usuarios.
- IP del cliente dentro de IpPermitida del usuario (si está configurada).
- Lista blanca seg.usuario_endpoints para usuarios no administradores (si tienen reglas activas).
- Permiso módulo «Actualizacion saldo y valor» + acción «EJECUTAR» (403).
- Misma consulta de candidatos que facturas_decimales.
- Aplica int(valor) por consecutivo_factura en lotes de 500.
- Commit al éxito; rollback ante error.

Entrada

Sin cuerpo.

Salida

200 — ApiExecutionResponse.

Flujo detallado

1. Validar permiso.
2. Obtener payload desde PostgreSQL.
3. Actualizar saldo_factura y valor_por_aplicar por lote.
4. Registrar log y devolver respuesta.

Códigos de respuesta

200 | 403 | 401 | 500 | 503

Ejecución (legacy)

POST /execute/service1

Alias legacy del ajuste decimal (oculto en OpenAPI).

Grupo OpenAPI

Autenticación

Autorización adicional

Content-Type

Auditoría

Ejecución (legacy)

Bearer JWT.

Igual que /execute/facturas_decimales.

—

Igual que facturas_decimales.

Tablas PostgreSQL (ERP Messiah)

Lectura: administrativo.sc_saldo_encabezado, administrativo.sc_saldo_detalle,
administrativo.sc_cuenta, administrativo.sc_tercero,
administrativo.sc_factura_encabezado, administrativo.ct_ips



EPS | FAMILIAR
DE COLOMBIA

Junto a ti

Tablas SQL Server (OrquestacionDB)

Lectura: orq.usuarios, seg.permisos, seg.perfiles, seg.usuario_perfil,
orq.resultados_procesos

Escritura: orq.resultados_procesos, orq.log_procesos

Validaciones

- Idénticas a POST /execute/facturas_decimales.

Entrada

Sin cuerpo.

Salida

200 — ApiExecutionResponse.

Flujo detallado

1. Validar permiso Facturas decimales.
2. Ejecutar OrchestratorService.run_service1.
3. Devolver respuesta.

Códigos de respuesta

200 | 403 | 401 | 500

5. Códigos HTTP habituales

200/201 — Éxito | 400 — Validación negocio | 401 — Token/login | 403 —
Permisos/ACL

404 — No encontrado | 409 — Conflicto | 422 — Esquema entrada | 500 — Error
interno | 503 — Servicio no disponible

Generado por scripts/generate_manual_flujos_endpoints.py — ORQUESTADORDB.